PROPOSAL 1 · QUICK BRIEF

Preventing Missing Records in Offline Khata Apps: A Study of Sync Methods in SmartOps

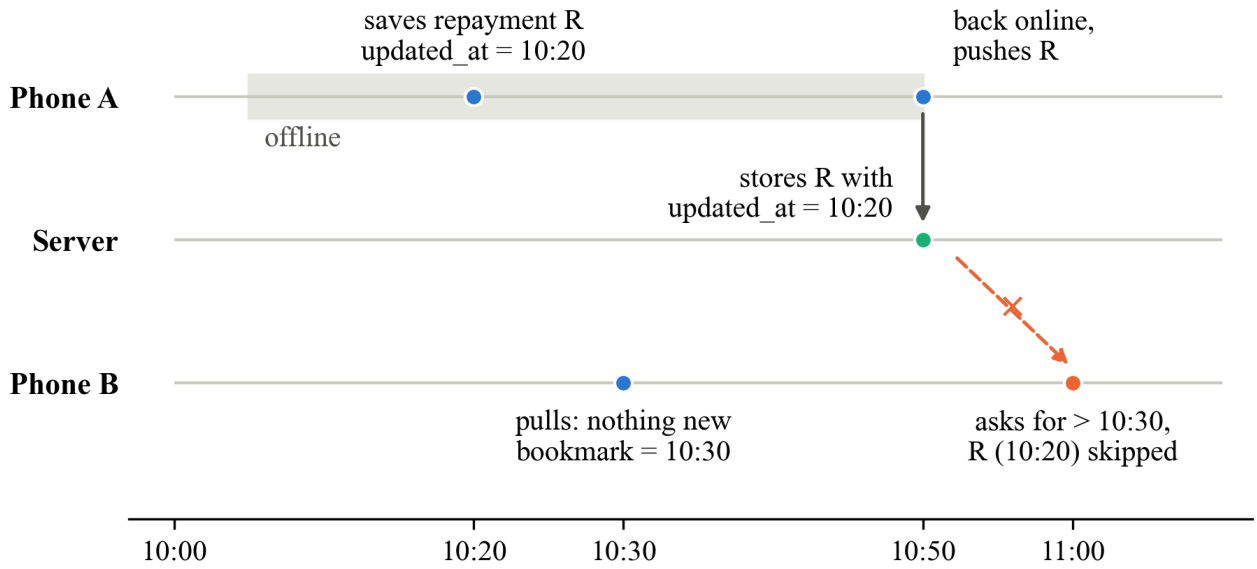
Research Methodology · what we are proposing, in plain words

In one line: our app can permanently "lose" a khata entry on the shop's second phone because of which time it uses while syncing. We will measure how often that happens and test two simple fixes.

1 The problem

SmartOps works offline. Every entry is saved on the phone first, and phones sync with the server later. When a phone syncs, it asks the server: "*give me everything newer than my last sync.*"

But "newer" is judged by the time written **on the phone that created the entry**, not the time the server received it. So an entry made offline and uploaded later looks "old" and gets skipped by another phone.



Phone A saves ₹500 repayment offline at 10:20 and uploads at 10:50. Phone B already synced at 10:30, so it only asks for records after 10:30 and never gets the 10:20 one.

What goes wrong

Phone B keeps showing Ramesh owes ₹500 more than he really does.

Where in our code

Phone stamps `updated_at` (actions.js:340); server filters `updated_at > last sync` (syncController.js:96).

Small extra issue

Phone keeps 99.999, server rounds to 100.00, so there's a paise-level gap.

2 What we are proposing

We compare **three ways** of deciding what a phone should download:

Current

Phone time

Record keeps the time it was written on the phone. This is the one with the bug.

Fix 1

Server timestamp

Server writes its own time when the record arrives. R becomes 10:50, so B gets it.

Fix 2

Server sequence number

Server gives each record the next number, like a bank token. B asks "after token 57", gets 58.

3 How we will test it

1. **Prove the bug** on two real phones (the exact Figure above).
2. **Add a switch** `SYNC_MODE` in our backend to pick one of the 3 methods.
3. **Write a Node.js script** that pretends to be 3 phones of one shop. It makes sales, credit and repayments and goes offline and online on a schedule.
4. **Run it 180 times:** 3 methods × 3 offline durations (5 min, 30 min, 2 h) × 20 runs. Time is sped up, so 1 second = 1 shop minute.
5. **After each run, compare** every phone with the server database and count what's missing.

%

Records missed

on the server but not on a phone

₹

Balance gap

how far phone balances are off

ms

Sync time

does the fix slow it down?

KB

Data size

does the fix use more data?

4 What we expect to show (hypotheses)

#	In plain words	Test	We are wrong if...
H1	The three methods miss different amounts. Phone time misses records, server methods almost none.	Chi-square	all three miss about the same
H2	With phone time, longer offline means more missed records.	Kruskal–Wallis	2 h offline isn't worse than 5 min
H3	The fixes don't make sync noticeably slower (under 10%).	Kruskal–Wallis	the fixes are clearly slower

Chi-square, in one breath

Count missed vs delivered records for each method, and check if the difference is too big to be luck.

Kruskal–Wallis, in one breath

Rank all runs from low to high and check if one group is consistently higher. No bell curve needed.

5 What we expect to find (expectations, not results)

- Phone time will miss records, and more of them with longer offline periods.
- Both server methods will miss almost none. A very rare miss can happen if a push and a pull hit the server in the same millisecond, and we'll count it honestly.
- The fixes cost very little extra time. There is slightly more data, because each phone re-downloads its own records once. That also removes the rounding gap.
- **Final output:** a recommended method for SmartOps, the fixed backend code, and a reusable test script.

6 Plan and cost

12 weeks
Weeks 1–3: trace and prove the bug · 3–6: build fixes and simulator · 7–9: run 180 tests · 9–12: analysis and report.

About ₹2,000
All free software; our own laptops and phones. The money is for printing, travel and a little mobile data. No people involved, all data is made up.

7 Say it in 30 seconds

PITCH

"Our khata app works offline and syncs later. It decides what other phones should download using the time on the phone that made the entry. So a repayment saved offline and uploaded later can be skipped forever by the shop's second phone, and the two phones then show different balances. We'll measure how often this happens with a script that acts as three phones against our real server, over 180 runs. Then we'll compare two fixes: server time and a server counter. We measure missed records, the rupee gap and the cost, and recommend one."

8 Tough questions, quick answers

"Isn't this just a bug fix?"
The fix is small. The research is measuring how often it happens, how it grows with offline time, how much money it affects, and what the fixes cost, all with a controlled test and hypotheses.

"Why not use Firebase?"
Paid, closed, and data sits on Google's servers. We use an open stack we control, and our question applies to anyone using pull/push sync.

"Is 20 runs enough?"
H1 counts records, about 10,000 per method, which is plenty. Kruskal–Wallis needs 5+ per group and we have 20. More runs are cheap if needed.

"Why not test in real shops?"
We can't control when phones go offline, it would take months, and it risks real money records. We still check the key case on two real phones.

"Isn't speeding up time cheating?"
No. The bug only depends on the *order*: A writes, B syncs, A uploads. Speeding up keeps the order.

Full proposal: Research_Proposal_1_Sync_Methods.pdf · Detailed viva notes: explainer_sync.md